COM2043 – PROGRAMMING LANGUAGES CONCEPTS

ASSIGNMENT: DESIGN AND IMPLEMENTATION OF A SIMPLE PROGRAMMING LANGUAGE

Topic: Lexical Analysis and Syntax Analysis Using Lex and Yacc

Deadline: 22 November 2026

1. Assignment Objective

The main objective of this assignment is to provide practical experience in **lexical analysis and syntax analysis** using Lex and Yacc tools.

In this assignment, you are expected to design and implement a simple programming language of your own. You will define the lexical and syntactic rules of your language, describe its grammar using Context-Free Grammar (CFG), and implement a syntax analyzer that checks whether a given source program conforms to your language specifications.

You will name your programming language **MPL (My Programming Language)**.

Rather than simply reproducing the syntax of an existing programming language such as C, C++, or Java, you are encouraged to introduce your own language design decisions and syntax conventions.

Your implementation must consist of:

- A Lex source file (mpl.l) for lexical analysis.
- A Yacc source file (mpl.y) for syntax analysis.
- Example source programs (.mpl) written in your programming language.

Important: This assignment focuses exclusively on lexical and syntactic analysis. You are NOT required to implement semantic analysis, type checking, code generation, or program execution.

2. Background and Preparation

Before starting the assignment, you should review the lecture materials related to lexical analysis, context-free grammars, Lex, and Yacc.

The following material is provided on the course homepage:

- Lex_and_Yacc_Tutorial.pdf
- Lex_Yacc.pdf
- Yacc_Tutorial.pdf

You are expected to understand the basic concepts of:

- Tokens, lexemes, and patterns.
- Regular expressions and regular grammars.
- Context-Free Grammars (CFG).
- BNF and EBNF notation.
- Lexical analyzer (Lex).
- Syntax analyzer (Yacc).
- Operator precedence and associativity.
- Syntax error detection and reporting.

3. Programming Language Design

You will design your own simple programming language called **MPL (My Programming Language)**.

First, you must determine the basic characteristics and syntax rules of your language. These rules must be clearly documented before implementing your lexical and syntax analyzers.

You have considerable freedom in designing your language. However, your design must satisfy all the minimum requirements specified in this assignment.

The following sections describe the minimum features that your language must support.

3.1. Program Structure

Define the general structure of an MPL program.

For example, you may decide that:

- Every program must start with the keyword begin.
- Every program must terminate with the keyword end.
- Every statement must be terminated by a semicolon (;).
- Comments may be defined using your own syntax.

Example:

```
begin
  int: counter;
  float: temperature;
  counter = 10;
  temperature = 25.5;
end
```

You may use a completely different program structure, provided that your rules are clearly defined and consistently implemented.

3.2. Data Types and Variable Declarations

Your language must support at least **three different data types**.

For example:

- int: Integer values.
- float: Floating-point values.
- char: Character values.

You must define the syntax of variable declarations.

For example:

```
int: number;
float: average, total;
char: initial;
```

You must also specify:

- How variable names are defined.
- Whether multiple variables can be declared in one statement.
- Whether initialization is allowed during declaration.
- What kinds of constants are supported.

Note: You are not required to check whether a variable has been declared before it is used. This is a semantic analysis issue and is outside the scope of this assignment.

3.3. Variable Names and Lexical Rules

You must define a well-structured lexical rule for variable names using regular expressions.

For example, you may define identifiers according to the following rules:

- An identifier must start with an alphabetic character.
- The remaining characters may contain letters, digits, or underscores.
- Identifiers cannot contain spaces.
- Reserved keywords cannot be used as variable names.

Example regular expression:

[a-zA-Z][a-zA-Z0-9_]*

You must also define the lexical rules for:

- Integer constants.
- Floating-point constants.
- Character constants.
- Keywords.
- Arithmetic and logical operators.
- Comparison operators.
- Separators and punctuation marks.

These rules must be implemented in your Lex source file (mpl.l).

3.4. Arithmetic Operators

Your language must support at least **four arithmetic operators**.

For example:

Operator	**Description**
+	Addition
-	Subtraction
*	Multiplication
/	Division

You must define the **precedence and associativity** of these operators.

For example:

- Multiplication and division have higher precedence than addition and subtraction.
- Operators with the same precedence level are left-associative.

Your language may also support additional operators, such as modulus (%), exponentiation (^), or unary minus.

3.5. Logical Operators

Your language must support at least **two logical operators**.

For example:

Operator	Description
&&	Logical AND
`	Logical OR
!	Logical NOT

You must specify their precedence and associativity rules.

3.6. Comparison Operators

Your language must support at least **two comparison operators**.

For example:

Operator	Description
==	Equal to
!=	Not equal to
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to

You must define how comparison operators interact with arithmetic and logical operators in expressions.

3.7. Conditional Statements

Your language must support at least **one conditional statement**.

You may design an if-else, switch-case, or another conditional structure.

For example:

```
if (number > 10) then
   result = number * 2;
else
   result = number + 5;
endif
```

You must define the complete syntax of the conditional statement, including:

- The condition.
- The statements executed when the condition is true.
- The optional or mandatory alternative branch.
- The keywords used to delimit the conditional structure.

3.8. Loop Structures

Your language must support at least **one loop structure**.

You may choose one or more of the following:

- for
- while
- do-while
- repeat-until

For example:

```
while (counter < 10) do
   counter = counter + 1;
endwhile
```

Alternatively, you may introduce your own loop syntax.

You must clearly define the grammar of the selected loop structure.

3.9. Expressions and Operator Precedence

Your language must support arithmetic, logical, and comparison expressions.

Expressions should allow multiple variables and constants combined using the supported operators.

For example:

result = (a + b) * (c - 5) / d;

You must explicitly define:

- Operator precedence.
- Operator associativity.
- The use of parentheses to change evaluation order.
- The syntax of unary operators, if supported.

Your precedence rules must be reflected in your Yacc implementation.

Important: Defining operator precedence only in the written report is not sufficient. Your Yacc grammar or precedence declarations must implement the same rules.

4. Minimum Technical Requirements

Your MPL language must satisfy ALL of the following minimum requirements:

1. A clearly defined program structure.
2. A well-defined regular grammar for variable names (identifiers).
3. At least three data types.
4. Clearly defined variable declaration rules.
5. At least four arithmetic operators.
6. At least two logical operators.
7. At least two comparison operators.
8. Explicit precedence and associativity rules for operators.
9. Support for arithmetic and logical expressions.
10. At least one conditional statement.
11. At least one loop structure.
12. A complete Context-Free Grammar (CFG) written in BNF or EBNF notation.
13. A working lexical analyzer implemented using Lex.

14. A working syntax analyzer implemented using Yacc.
15. Successful syntax checking of both valid and invalid MPL source programs.

Originality Requirement: Your programming language should demonstrate your own design decisions. Simply copying the syntax of C or another existing programming language without introducing meaningful modifications is not acceptable.

You are encouraged to introduce additional language features, keywords, operators, or alternative syntax structures beyond the minimum requirements.

5. Implementation Using Lex and Yacc

After defining your language specifications and writing its CFG, you must implement the lexical and syntax analyzers.

Your implementation must contain the following files:

5.1. Lex Source File – mpl.l

The Lex source file is responsible for identifying tokens in the input source program.

It should recognize:

- Keywords.
- Identifiers.
- Numeric and character constants.
- Arithmetic operators.
- Logical operators.
- Comparison operators.
- Assignment operators.
- Separators and punctuation marks.

Whitespace and comments should be handled appropriately according to your language design.

5.2. Yacc Source File – mpl.y

The Yacc source file is responsible for checking whether the sequence of tokens generated by Lex conforms to the grammar of MPL.

It should include:

- Grammar rules for program structure.
- Grammar rules for variable declarations.

- Grammar rules for assignments.
- Grammar rules for expressions.
- Grammar rules for conditional statements.
- Grammar rules for loop structures.
- Operator precedence and associativity declarations, where appropriate.
- Syntax error handling.

You do not need to perform any semantic checks or execute the statements in the source program.

5.3. Example Source Programs

You must prepare example MPL source programs to demonstrate that your analyzer works correctly.

At least:

- One valid MPL program containing the required language features.
- One or more invalid MPL programs containing deliberate syntax errors.

You may create separate files for each test case.

6. Compilation and Execution

You can compile and execute your lexical and syntax analyzers using the following commands:

Step 1 – Generate the Lex analyzer:

lex mpl.l

Step 2 – Generate the Yacc parser:

yacc mpl.y

Step 3 – Compile the generated C source file:

gcc -o mpl y.tab.c -lfl

Step 4 – Execute the syntax analyzer:

./mpl < myprog.mpl

If your implementation is correct, the following behavior is expected:

- If the input program conforms to your MPL grammar, the analyzer should display:

OK

- If the input program contains a syntax error, the analyzer should display an appropriate error message, such as:

Syntax error

You may use a more informative error message if you wish.

Note: Depending on your system configuration, you may need to use flex instead of lex, or bison -y instead of yacc. Make sure that your compilation commands are compatible with the environment you use.

7. Deliverables

You must submit the following files:

1. Technical Report

- A PDF file describing your language design, grammar, implementation, and test results.

2. Source Code

- mpl.l
- mpl.y

3. Example Programs

- At least one valid .mpl source program.
- At least one invalid .mpl source program.

4. Execution Results

- Screenshots or other evidence demonstrating the successful execution of your analyzer on valid and invalid input programs.

Submit all files together in a single compressed archive.

Two tutorials will be held for this project on **November 7** and **November 14**.

Students may form groups of **two** to work on the project. However, students who prefer to work individually may also complete the project on their own.

Within **two weeks after the project submission deadline**, each group (or individual student) is required to give a presentation of their project.